

DOCUMENTATION TECHNIQUE

Du support de formation au cours e-learning

Architecture, algorithmes et décisions d'implémentation

Réalisé par	Hala Sadoumi
Structure d'accueil	S2M Casablanca
Filière	4ème année — Intelligence Artificielle & Data Science
Établissement	École Marocaine des Sciences de l'Ingénieur, Casablanca
Objet	Parcours essentiel — chaîne PDF vers cours
Année universitaire	2025 / 2026

Résumé

Ce document décrit la chaîne qui transforme un support de formation au format PDF en cours e-learning : huit chapitres de moins de trois minutes, narration synthétisée, schémas animés, sous-titres exacts, évaluation et publication.

La décision structurante est de ne pas écrire une seconde chaîne de traitement. Le support est converti dans la représentation interne que produisait déjà l'ingestion vidéo — une Scene par page — après quoi six étages sur neuf fonctionnent sans modification. Ajouter cette source entièrement nouvelle a demandé trois modules Python et six composants de rendu.

Le document expose chaque étage par son principe, son algorithme, ses paramètres et son code, avec les mesures relevées sur la production réelle. Les décisions sont présentées avec leurs raisons, y compris les tentatives qui se sont révélées fausses : une consigne trop large qui faisait recopier les diapositives au lieu de les enseigner, un découpage qui déséquilibrait les chapitres, une graine prétendument déterministe qui ne l'était pas, et des valeurs décimales arrondies à l'image.

Il se termine par les propriétés du système — reproductibilité, résilience, couverture de tests — ses limites connues, et la marche à suivre pour lui ajouter une troisième source ou un nouveau schéma animé.

Table des matières

1	Objet du document.....	4
1.1	Ce qui a motivé cette chaîne.....	4
1.2	Le résultat.....	4
2	Architecture.....	6
2.1	Le principe : adapter l'entrée, pas la chaîne.....	6
2.2	Les trois zones du paquet.....	6
2.3	Le flux de données.....	7
2.4	L'orchestrateur.....	8
3	Étage 1 — Lire le support.....	9
3.1	Extraction des pages.....	9
3.2	Détection des sections.....	9
3.3	Groupement en chapitres.....	10
4	Étage 2 — Rédiger la narration.....	12
4.1	Une lecture multimodale.....	12
4.2	La sortie contrainte par schéma.....	12
4.3	La consigne, et sa correction.....	13
4.4	Écrire pour une voix de synthèse.....	13
4.5	Le choix du modèle et la gestion du quota.....	14
5	Étages 3 à 5 — Scènes, voix et sous-titres.....	15
5.1	Découpage en scènes.....	15
5.2	Synthèse vocale.....	15
5.3	Le plafond de cinq minutes, vérifié.....	16
5.4	Sous-titres calés sur la parole.....	16
6	Étage 6 — Le plan visuel.....	17
6.1	Un vocabulaire fermé d'archétypes.....	17
6.2	Ce que porte chaque scène.....	17
6.3	La revalidation des réponses.....	18
6.4	La diversification.....	18
6.5	Les six gabarits ajoutés.....	19
7	Étages 7 et 8 — Images et rendu.....	21
7.1	Les arrière-plans.....	21
7.2	Le rendu vidéo.....	21
8	Étage 9 — Publication.....	23
8.1	Le quiz officiel.....	23
8.2	La plateforme de consultation.....	23
9	Propriétés du système.....	25
9.1	Reproductibilité.....	25
9.2	Résilience.....	25
9.3	Les tests.....	26
10	Limites connues et suites.....	27
10.1	Ce qui n'est pas résolu.....	27
10.2	Étendre le système.....	27
10.3	Ajouter un schéma animé.....	28

1 Objet du document

Ce document décrit, dans le détail de son implémentation, la chaîne qui transforme un support de formation au format PDF en un cours e-learning structuré : chapitres courts, narration synthétisée, schémas animés, sous-titres, évaluation et publication sur une plateforme de consultation.

Il s'adresse à un lecteur technique. Chaque étape est présentée par son principe, son algorithme, ses paramètres et le code qui la met en œuvre, avec les mesures relevées sur la production réelle. Les décisions de conception sont exposées avec leurs raisons, y compris lorsque la première tentative s'est révélée fausse — ces échecs sont souvent la partie la plus instructive.

1.1 Ce qui a motivé cette chaîne

Le système existait déjà pour une autre entrée : l'enregistrement vidéo d'une session de formation. Il en produisait un cours de dix-sept chapitres et quatre-vingts minutes, fidèle à la session mais long, certains chapitres atteignant dix minutes.

La demande a changé de nature. Il ne s'agissait plus de restituer une session mais de produire un cours court à partir du support pédagogique : des chapitres de moins de cinq minutes, allant droit au but, sans référence à la réunion d'origine. Une exigence s'est ajoutée en cours de route et a orienté toute la rédaction : chaque chapitre doit contenir l'intégralité de ce que l'apprenant doit savoir sur son point. Court ne signifie pas incomplet.

Les deux cours coexistent désormais et sont présentés comme deux lectures de la même formation : un parcours détaillé pour approfondir, un parcours essentiel pour aller à l'essentiel.

1.2 Le résultat

Grandeur	Valeur mesurée
Pages de support en entrée	28
Mots de texte extraits des diapositives	1 405
Mots de narration produits	2 625
Rapport narration / source	1,87

Chapitres	8
Durée totale du cours	17,4 minutes
Durée par chapitre	de 1,4 à 3,2 minutes
Scènes animées	68 (15,3 s en moyenne)
Archétypes visuels distincts employés	17
Répétitions consécutives d'archétype	0
Sous-titres	110
Questions d'évaluation	8 (quiz officiel)
Durée de production sur poste sans carte graphique	environ 1 h 30

Tableau 1.1 — Le cours produit, en chiffres relevés sur les artefacts.

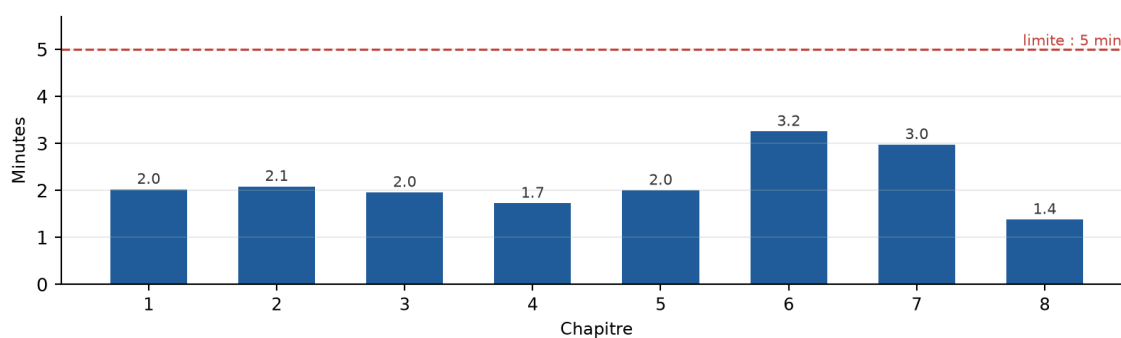


Figure 1.1 — Durée mesurée de chaque chapitre après synthèse vocale. La limite de cinq minutes n'est pas approchée.

2 Architecture

2.1 Le principe : adapter l'entrée, pas la chaîne

La décision structurante de ce travail tient en une phrase : plutôt que d'écrire une seconde chaîne de traitement pour le support PDF, on fait ressembler le support à ce que la chaîne existante sait déjà consommer.

L'ingestion vidéo produit une liste d'objets Scene : un identifiant, un intervalle temporel, le texte affiché à l'écran, le chemin d'une image représentative. Le nouvel étage de lecture du PDF produit exactement la même chose — une Scene par page, dont le texte est celui de la diapositive et dont l'image est un rendu de la page.

Point clé

À partir de cette conversion, les six étages suivants — plan visuel, arrière-plans, rendu, sous-titres, quiz, publication — fonctionnent sans une seule modification. Ce n'est pas une économie de code : c'est ce qui garantit que les deux cours partagent la même qualité de rendu et les mêmes corrections.

Le prix de ce choix est une convention un peu artificielle : les pages sont posées sur une ligne de temps synthétique de dix secondes par page. Cette ligne n'a aucune signification physique ; elle sert uniquement à ce que la logique de recouvrement temporel des étages aval continue de fonctionner sans cas particulier. La ligne de temps réelle du cours est calculée à l'assemblage, à partir des durées mesurées des vidéos rendues.

2.2 Les trois zones du paquet

Le paquet Python est rangé selon un critère unique : un module connaît-il la provenance du contenu ?

src/s2m_pipeline/

config.py	parametres, lus par tout le monde
models.py	structures de donnees partagees
core/	commun aux deux entrees
llm.py	appels au modele, invites, schemas de sortie
audio.py	extraction, amelioration, decoupe, conversion
scene_visuals.py	archetype, points et phrase par scene
scene_images.py	arriere-plans generes
chapter_subtitles.py	appariement et ecriture des sous-titres
quiz.py	generation des questions
quiz_reference.py	lecture d'un quiz officiel fourni en Word
assemble.py	concatenation et metadonnees de navigation
web_export.py	publication vers la plateforme

```

from_video/      entree << enregistrement de session >>
  transcription.py, scenes.py, ocr.py, diarization.py
  embeddings.py, chaptering.py
  content_selection.py, narration_original.py
  build_course.py                                orchestrateur

from_slides/      entree << support de formation >>
  pdf_source.py      lecture du document
  storyboard.py      decoupage en scenes
  narration.py, tts.py      synthese vocale
  build_course_from_pdf.py      orchestrateur

```

Neuf modules sont dans `core/` et servent aux deux parcours. Les imports internes sont absolus — `from s2m_pipeline.core import llm` — plutôt que relatifs : un lecteur voit d'où vient chaque chose sans avoir à compter les points ni à savoir où il se trouve dans l'arborescence.

Ce qu'il a fallu écrire pour cette entrée	Nature
<code>pdf_source.py</code>	nouveau — lecture et découpage du support
<code>build_course_from_pdf.py</code>	nouveau — orchestrateur des neuf étapes
<code>quiz_reference.py</code>	nouveau — lecture d'un quiz officiel Word
<code>storyboard.py</code> , <code>narration.py</code> , <code>tts.py</code>	existants, gardés en réserve depuis des semaines
6 composants de rendu	nouveaux gabarits visuels
<code>core/</code> , <code>from_video/</code> , le reste de <code>remotion/</code>	inchangés

Tableau 2.1 — Coût réel de l'ajout d'une source entièrement nouvelle.

2.3 Le flux de données

Chaque étage lit des fichiers et en écrit d'autres. Aucun état n'est conservé en mémoire d'une étape à l'autre. Ce choix a un coût — sérialisation et désérialisation systématiques — et trois bénéfices déterminants : chaque décision du système est lisible dans un fichier, une exécution interrompue reprend là où elle s'est arrêtée, et remplacer un composant revient à écrire un module produisant le même fichier.

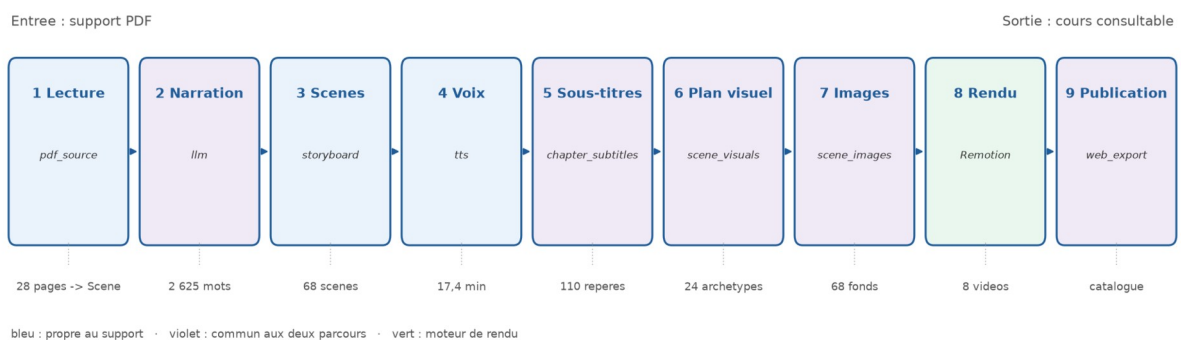


Figure 2.1 — Les neuf étapes, le module qui les met en œuvre et ce que chacun a produit sur le support de validation.

Étape	Fichier produit	Contenu
1 — Lecture	pages.json	une Scene par page : texte, image, intervalle
2 — Narration	chapters.json, scripts.json	titre, résumé, points clés, texte à lire
3 — Découpage	storyboard.json	scènes avec leur extrait de narration
4 — Voix	work/narration/*.wav, narration_marks.json	audio et repères de phrase
5 — Sous-titres	subtitles/*.vtt	une piste par chapitre
6 — Plan visuel	scene_visuals.json	archétype, libellés, icône, invite d'image
7 — Images	work/backdrops/*.jpg	un arrière-plan par scène
8 — Rendu	remotion/out_pdf/*.mp4	une vidéo par chapitre
9 — Publication	web/data/<cours>/	vidéos, sous-titres, quiz, métadonnées

Tableau 2.2 — Les contrats entre étapes : ce que chacun laisse au suivant.

2.4 L'orchestrateur

Une seule commande enchaîne les neuf étapes. Chacune vérifie d'abord si son résultat existe déjà et se saute le cas échéant, ce qui rend l'exécution reprenable après une coupure, un quota d'API épuisé ou un arrêt volontaire.

Production complète d'un cours à partir d'un support

```
.venv\Scripts\Activate.ps1

s2m-course-pdf --pdf "support.pdf" \
               --course-id mon_cours \
               --title "Titre du cours" \
               --quiz-docx "quiz.docx"      # facultatif
```

Un défaut de reprise trouvé en production

La première version jugeait une étape terminée à la simple existence de son fichier. Une exécution arrêtée à sept chapitres sur huit laissait donc un chapters.json qui ressemblait à un travail fini, et le huitième chapitre n'aurait jamais été rattrapé.

La complétude se juge désormais sur le nombre de groupes de pages attendus, pas sur la présence du fichier. C'est le genre de défaut qui ne se voit qu'en le provoquant.

3 Étage 1 — Lire le support

Module : `from_slides/pdf_source.py`. Cet étage transforme un document en la représentation interne que le reste de la chaîne consomme, et détermine le découpage en chapitres.

3.1 Extraction des pages

Chaque page produit deux choses : son texte, extrait de la couche textuelle du PDF, et un rendu de la page en image à 110 points par pouce. Les deux comptent : sur ce support, quatre diapositives ne contiennent presque aucun texte et portent toute leur information dans des graphiques.

`pdf_source.extract_pages`

```
for index, page in enumerate(document):
    image_path = image_dir / f"page_{index:03d}.png"
    if not image_path.exists():
        page.get_pixmap(dpi=_PAGE_DPI).save(image_path)

    scenes.append(Scene(
        id=f"page_{index:03d}",
        start=index * PAGE_SECONDS,          # ligne de temps synthétique
        end=(index + 1) * PAGE_SECONDS,
        ocr_text=" ".join(page.get_text().split()),
        frame_path=str(image_path),
    ))
```

Le champ s'appelle `ocr_text` alors qu'aucune reconnaissance optique n'a lieu ici : le texte est lu directement dans le PDF. Le nom est conservé parce qu'il est celui du contrat avec les étages suivants, qui ne doivent pas savoir d'où vient le texte.

3.2 Détection des sections

Un support de formation est structuré par des pages de séparation qui n'annoncent qu'un titre de partie. Les reconnaître permet de ne jamais faire chevaucher un chapitre sur deux parties du document.

La règle retenue ne connaît pas le sujet : une page de séparation fait moins de soixante-dix caractères et est écrite à plus de quatre-vingts pour cent en capitales.

`pdf_source.is_section_divider`

```
_DIVIDER_MAX_CHARS = 70
_DIVIDER_MIN_UPPERCASE = 0.8

def is_section_divider(text: str) -> bool:
    clean = " ".join(text.split())
```

```
return (bool(clean)
        and len(clean) <= _DIVIDER_MAX_CHARS
        and _uppercase_ratio(clean) >= _DIVIDER_MIN_UPPERCASE)
```

Page	Texte	Capitales	Verdict
3	DÉFINITIONS ET PRINCIPES DE BASE	100 %	séparation
9	ÉVOLUTION DE LA CYBERCRIMINALITÉ	100 %	séparation
15	IMPACT DE LA CYBERCRIMINALITÉ	100 %	séparation
22	DÉPLOIEMENT DE LA CYBERSÉCURITÉ	100 %	séparation
28	Sécurité SI... Questions & Réponses	12 %	contenu

Tableau 3.1 — La règle appliquée au support : quatre sections trouvées, la page de clôture correctement écartée.

Le plan interne du PDF est utilisé en priorité lorsqu'il existe, puisqu'il exprime l'intention de l'auteur. Ce support n'en a pas, et la plupart des exports de diaporama non plus, d'où l'importance de la règle de repli.

3.3 Groupement en chapitres

Le nombre de chapitres n'est pas un paramètre : il découle du volume du support. Le système vise environ cent quatre-vingt-dix mots de source par chapitre, valeur exprimée comme un objectif dans la configuration et non comme une constante réglée sur ce document.

src/s2m_pipeline/config.py

```
pdf_chapter_target_words: int = 190
pdf_chapter_min_words: int = 70
chapter_max_seconds: float = 300.0 # plafond verifie apres synthese
```

Chaque section est ensuite divisée en autant de chapitres que son volume le justifie, et le découpage cherche l'équilibre plutôt que le remplissage.

pdf_source.group_into_chapters — decision de fermeture

```
count = max(1, round(total / settings.pdf_chapter_target_words))
count = min(count, len(section))
per_chapter = total / count

accumulated = sum(_words(s) for s in current)
next_words = _words(section[index + 1]) if pages_left else 0
close = (chapters_left > 0
         and pages_left >= chapters_left
         and abs(accumulated - per_chapter)
            <= abs(accumulated + next_words - per_chapter))
```

Un défaut trouvé par les tests, pas par l'œil

La première version fermait un chapitre dès que sa part était atteinte. Comme une page pèse cent mots et la part cible deux cents, elle dépassait systématiquement : une section de six cent deux mots sortait en 202 / 300 / 100 au lieu de trois tiers.

La condition compare désormais deux distances : est-on plus près de la cible en s'arrêtant maintenant, ou en prenant la page suivante ? Le découpage du support réel est passé de [30, 176, 147, 180, 151, 248, 354, 119] à [30, 176, 147, 180, 151, 248, 204, 269].

4 Étape 2 — Rédiger la narration

Module : `core/llm.py`, fonction `generate_slide_chapter`. C'est l'étage qui produit le contenu pédagogique proprement dit.

4.1 Une lecture multimodale

Le modèle reçoit, pour chaque chapitre, le texte des diapositives et jusqu'à six images de pages. Les images ne sont pas un supplément : elles portent souvent l'essentiel.

Point clé

Sur la diapositive « Ausimètre de la cybersécurité au Maroc », le texte extrait ne contenait qu'une phrase. Les chiffres — 84 % de rançongiciels, 61 % de fuites de données, 32 % des répondants au-dessus de 500 000 dirhams, 6 % au-delà de 10 millions — étaient tous dans des encadrés graphiques. Sans lecture de l'image, ils étaient perdus.

Vérification faite : chacun de ces chiffres cité dans la narration figure bien sur la diapositive. Le modèle a lu le graphique, il ne l'a pas inventé.

4.2 La sortie contrainte par schéma

La difficulté d'intégrer un modèle de langage dans une chaîne automatisée n'est pas la qualité du texte mais sa forme. Un modèle sollicité en texte libre peut répondre correctement tout en encadrant sa réponse d'une phrase d'introduction ou en produisant un JSON invalide.

Le décodage contraint résout cela à la racine : un schéma est transmis avec la requête, et les jetons qui rendraient la sortie non conforme sont écartés avant échantillonnage. Le modèle ne peut littéralement pas produire une réponse mal formée.

`core/llm.py`

```
class SlideChapterContent(BaseModel):
    title: str
    summary: str
    key_points: list[str]
    narration: str

response = _generate_content(
    client, model=settings.gemini_model_writing, contents=contents,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=SlideChapterContent,      # contrainte de forme
    ),
)
```

Le schéma sert deux fois : il contraint la génération côté modèle et valide la réponse côté programme. Les deux usages reposent sur la même déclaration, ce qui exclut toute divergence.

4.3 La consigne, et sa correction

Le premier jet a produit un rapport narration sur source de 1,08 : le modèle recopiait les diapositives. L'instruction « n'invente rien » était trop large et il l'avait interprétée comme « reste au plus près du texte ».

La correction distingue deux choses que la formulation initiale confondait :

- Inventer un fait, un chiffre, une date, un nom ou une norme absent des diapositives est interdit.
- Expliquer, reformuler, illustrer et relier les idées est exactement ce qu'on demande — c'est cela, enseigner.

Une seconde correction a suivi, lorsqu'une vérification a montré que la narration disait « les fuites de données et la compromission des messageries » là où le support précisait 61 % et 45 %. La consigne pose désormais explicitement la densité avant la brièveté.

Extrait de l'invite `_SLIDE_CHAPTER_PROMPT`

Le critère n'est pas la longueur, c'est la DENSITE. Un chapitre doit rester court ET contenir tout ce que l'apprenant doit savoir sur ce point.

Ce qui se coupe : les tournures de remplissage, les redites, les phrases de liaison qui n'apportent rien, les adjectifs décoratifs.

Ce qui ne se coupe JAMAIS : les données. Chaque pourcentage, montant, date, durée, classement et source nommée présents sur les diapositives -- y compris ceux qui ne sont que dans un graphique ou un encadre de l'image -- doit figurer dans la narration. Un chiffre coûte trois mots : il n'y a aucune raison de le sacrifier.

Mesure	Avant	Après
Rapport narration / source	1,08	1,87
Chiffres cités, diapositive PwC	3 sur 8	7 sur 8
Chapitres sans données (mots)	296, 316, 306	281, 301, 287
Chapitres avec données (mots)	334, 349	553, 441

Tableau 4.1 — L'effet de la consigne de densité : les chapitres sans chiffres ont maigri, ceux qui en portent se sont étoffés.

4.4 Écrire pour une voix de synthèse

Le texte sera lu par un moteur vocal, ce qui impose des contraintes de forme absentes d'un texte destiné à la lecture :

- Phrases courtes, une idée par phrase, peu d'incises et pas de parenthèses.
- Les nombres écrits comme ils se prononcent : « quatre-vingt-quatre pour cent » plutôt que « 84 % », que le moteur lirait mal.
- Les sigles développés à leur première apparition.
- Les accents et diacritiques corrects, faute de quoi la prononciation dérive.

4.5 Le choix du modèle et la gestion du quota

Deux modèles sont employés. Les tâches mécaniques — découpage en scènes, plan visuel, quiz — utilisent un modèle allégé dont le quota gratuit est généreux. La rédaction de la narration, elle, utilise un modèle plus complet, parce que le modèle allégé perd régulièrement les accents et écrit une prose plate.

src/s2m_pipeline/config.py

```
gemini_model: str = os.getenv("GEMINI_MODEL", "gemini-flash-lite-latest")
gemini_model_writing: str = os.getenv("GEMINI_MODEL_WRITING", "gemini-3.6-flash")
```

Le modèle de rédaction est plafonné à vingt requêtes par jour en offre gratuite. Ce plafond a été atteint en production, et l'étape échouait sèchement. Elle bascule désormais sur le modèle allégé en le signalant, plutôt que de perdre l'exécution.

core/llm.py — repli sur quota epuise

```
try:
    response = call(settings.gemini_model_writing)
except genai_errors.APIError as exc:
    if exc.code != 429:
        raise
    print(f"    quota epuise, repli sur {settings.gemini_model}")
    response = call(settings.gemini_model)
```

5 Étages 3 à 5 — Scènes, voix et sous-titres

5.1 Découpage en scènes

Module : `from_slides/storyboard.py`. La narration d'un chapitre est découpée en scènes de dix à vingt-cinq secondes, à des frontières de phrases. Chaque scène recevra son propre schéma animé.

Le découpage est confié au modèle avec une contrainte forte : l'extrait de narration doit être reproduit exactement, et les extraits mis bout à bout doivent reconstituer le script intégral. Aucune réécriture n'est tolérée à cette étape — le texte a déjà été validé et sera lu tel quel.

Sur ce cours : 68 scènes, de 15,3 secondes en moyenne.

5.2 Synthèse vocale

Modules : `from_slides/tts.py` et `narration.py`. Il n'existe aucun enregistrement du support ; la narration passe donc par la synthèse. La voix retenue est fr-FR-DeniseNeural, via `edge-tts` — gratuit et sans clé d'API.

Point clé

La durée de chaque scène est mesurée sur le fichier audio produit, jamais estimée à partir du nombre de mots. C'est cette durée qui détermine le nombre exact d'images à rendre, donc la synchronisation entre la voix et l'animation, et c'est elle qui permet de vérifier le plafond de cinq minutes.

Le moteur produit du MP3 alors que le moteur de rendu charge des WAV. La conversion est faite dans `audio.to_wav` plutôt qu'en apprenant un second format au moteur de rendu : le contrat reste à un seul endroit.

`from_slides/build_course_from_pdf.py` — *synthese incrementale*

```
for scene in tqdm(scenes, desc=" synthesising"):
    mp3 = paths.narration_dir / f"{scene.scene_id}.mp3"
    wav = paths.narration_dir / f"{scene.scene_id}.wav"
    # Ne pas refaire ce qui est deja dit : une nouvelle synthese decalerait
    # la duree de quelques centiemes et desynchroniserait un chapitre deja
    # rendu contre l'ancienne valeur.
    if not (wav.exists() and scene.scene_id in marks):
        marks[scene.scene_id] = tts.synthesize_with_marks(scene.narration, mp3)
        audio_mod.to_wav(mp3, wav)
```

5.3 Le plafond de cinq minutes, vérifié

Une fois les durées connues, le système contrôle chaque chapitre contre le plafond configuré et signale tout dépassement. La contrainte n'est pas seulement espérée par la consigne de rédaction : elle est vérifiée sur le produit fini.

`check_chapter_lengths`

```
for chapter in chapters:
    seconds = by_chapter.get(chapter.id, 0.0)
    flag = "" if seconds <= settings.chapter_max_seconds else " DEPASSE LA LIMITE"
    print(f" {chapter.id}: {seconds / 60:4.1f} min {chapter.title}{flag}")
```

5.4 Sous-titres calés sur la parole

Le moteur vocal renvoie, pendant la synthèse, un repère par phrase : l'instant précis où chaque phrase est prononcée et sa durée. Les sous-titres sont construits sur ces repères.

`from_slides/tts.py`

```
async for chunk in communicate.stream():
    if chunk["type"] == "audio":
        handle.write(chunk["data"])
    elif chunk["type"] in ("SentenceBoundary", "WordBoundary"):
        # Les decalages arrivent en unites de 100 nanosecondes.
        marks.append({
            "start": chunk["offset"] / 10_000_000,
            "end": (chunk["offset"] + chunk["duration"]) / 10_000_000,
            "text": chunk["text"],
        })
```

Point clé

Les sous-titres sont donc exacts par construction. Il n'y a aucune estimation, donc aucune dérive possible — le défaut qui avait affecté la première version du parcours détaillé, où le texte était réparti uniformément sur la scène.

110 sous-titres pour ce cours. Ils servent deux usages : la piste activable sur la vidéo, et la transcription navigable affichée sous le lecteur.

6 Étape 6 — Le plan visuel

Module : `core/scene_visuals.py`. C'est l'étage qui décide de ce que l'apprenant voit.

6.1 Un vocabulaire fermé d'archétypes

Chaque scène se voit attribuer un archétype parmi vingt-quatre, chacun adossé à un composant de rendu qui s'anime seul. Le modèle ne décrit pas un visuel en texte libre : il choisit un identifiant dans une liste fermée et remplit ses emplacements de texte.

Les définitions transmises au modèle sont neutres de domaine — elles décrivent une forme de raisonnement, jamais un sujet :

`core/llm.py` — extrait de `SCENE_ARCHETYPES`

```
"pillars":      "2 a 4 fondements nommes qui s'elevent"
"actor_action_target": "un acteur agit sur une cible, avec un effet"
"bar_chart":     "PLUSIEURS valeurs mesurees que l'on compare"
"donut_share":  "UNE proportion rapportee a un tout"
"ranking_list":  "un classement ordonne, le rang porte l'information"
"venn_overlap":  "une notion contenue dans une autre"
"funnel":        "des etapes qui filtrent, chacune ne laisse passer qu'une partie"
"pyramid":       "des niveaux empiles, la base porte le sommet"
```

Aucune ne mentionne la sécurité informatique. C'est cette neutralité qui permet au même vocabulaire de servir une formation à la conformité, à un processus interne ou à un produit.

6.2 Ce que porte chaque scène

Champ	Rôle
archetype	l'identifiant du schéma à dessiner
label	titre de la scène, 2 à 6 mots
items	2 à 4 points porteurs de sens, obligatoires
primary / secondary	l'élément central et sa précision, selon l'archétype
takeaway	une phrase énonçant ce que l'apprenant doit retenir
icon	une icône du vocabulaire installé
image_prompt	la description, en anglais, de l'arrière-plan à générer

Tableau 6.1 — Les emplacements de texte d'une scène.

Une correction majeure héritée du parcours détaillé

Cinq archétypes recevaient auparavant la consigne explicite de laisser leur liste vide, ce qui laissait 141 scènes sur 261 avec un titre et rien d'autre — 54 % du cours. Un apprenant coupant le son n'apprenait presque rien.

Les points sont désormais obligatoires sur tous les archétypes, et le champ takeaway a été ajouté. Sur ce cours : zéro scène sans contenu à l'écran.

6.3 La revalidation des réponses

Le schéma de sortie contraint la forme, jamais le fond. Un champ de type chaîne n'empêche pas le modèle d'y écrire un identifiant d'archétype inexistant. Le code revalide donc systématiquement contre le vocabulaire et se rabat sur une valeur sûre.

Le modèle omet aussi parfois une scène dans un lot. Ces scènes tombaient sur une carte de repli affichant la narration tronquée au soixante-dixième caractère, en plein mot — visible dans la vidéo finie. Elles font maintenant l'objet d'un second appel ciblé, et le repli coupe sur une frontière de mot.

6.4 La diversification

Le planificateur juge chaque scène isolément : il ne voit pas le chapitre entier et ne peut donc pas savoir qu'il se répète. Mesuré avant correction : neuf répétitions consécutives sur soixante transitions, dont quatre chronologies à la file et quatre cartes typographiques à la file.

Chaque choix était défendable pris isolément ; bout à bout, cela se lisait comme une seule diapositive répétée. La règle est donc appliquée par le code, après coup.

core/scene_visuals.py

```
_INTERCHANGEABLE = (
    ("checklist", "pillars", "hierarchy", "pyramid", "concentric_layers"),
    ("timeline", "funnel", "cycle"),
    ("bar_chart", "stat_row", "ranking_list"),
    ("stat_reveal", "donut_share"),
    ("title_statement", "quote_highlight"),
    ("comparison", "separated_groups", "do_dont"),
)
_RECENT_WINDOW = 3

def diversify(plans: list[dict]) -> int:
    seen, changed = [], 0
    for plan in plans:
        archetype = plan.get("archetype", "")
        recent = seen[_RECENT_WINDOW:]
        if archetype in recent:
            for candidate in _alternatives(archetype, plan):
                if candidate not in recent:
                    plan["archetype"] = candidate; archetype = candidate
                    changed += 1; break
        seen.append(archetype)
    return changed
```

Les substitutions restent à l'intérieur de groupes de même forme : une séquence ne devient pas un ensemble, une proportion ne devient pas un classement. Un second niveau autorise, pour une scène portant au moins deux points nommés, tout schéma de type liste — ce qui donne à la passe où aller sur une longue série de cartes identiques.

Chapitre	Avant	Après
4	timeline timeline timeline timeline	timeline funnel cycle checklist
6	bar_chart bar_chart bar_chart	bar_chart stat_row ranking_list
7	title_statement × 4	title_statement quote_highlight checklist pillars

Tableau 6.2 — Trois séries cassées par la diversification.

Point clé

Résultat sur le cours : vingt scènes redessinées, zéro répétition consécutive, dix-sept archétypes distincts employés.

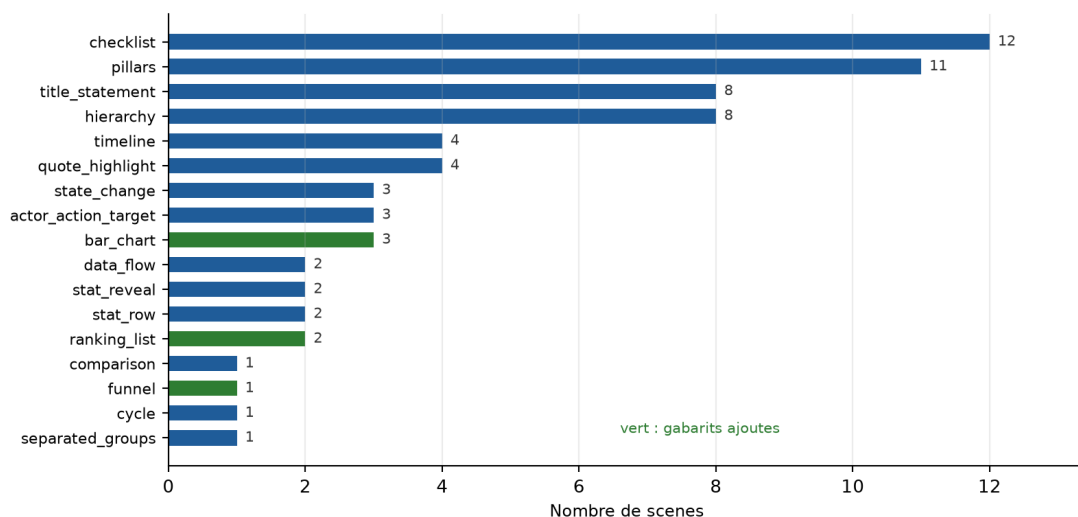


Figure 6.1 — Répartition des archétypes sur les 68 scènes. En vert, les six gabarits ajoutés pour cette production.

6.5 Les six gabarits ajoutés

Le vocabulaire d'origine plaçait surtout du texte dans des boîtes. Un chapitre citant des chiffres a besoin d'une forme qui montre une grandeur, pas seulement qui la nomme.

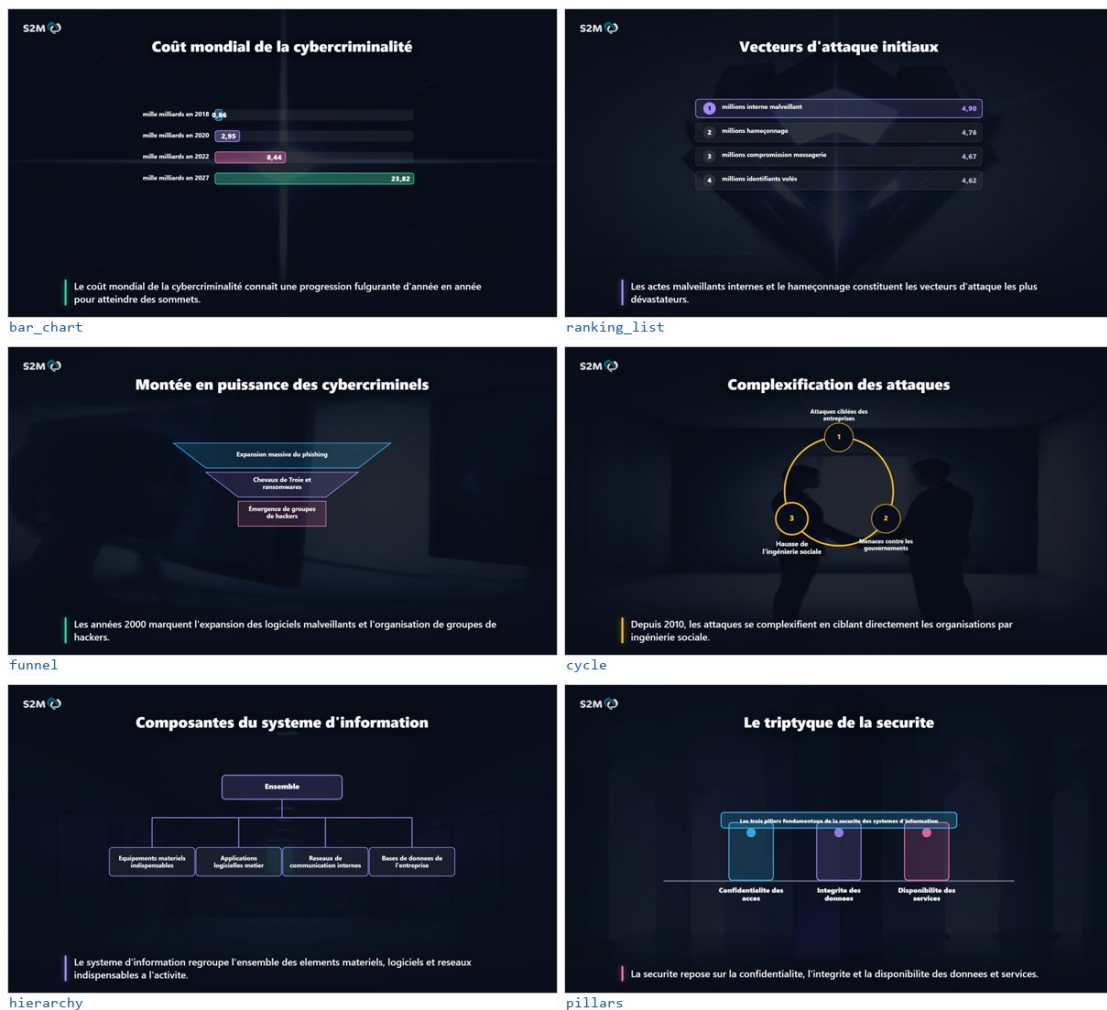
Un défaut de justesse trouvé à l'image

Les valeurs décimales étaient arrondies à l'entier : « 0,86 mille milliards » s'affichait 1, « 10,93 millions » s'affichait 11. Les chiffres portés à l'écran ne correspondaient plus au document dont ils venaient.

Le composant conserve désormais la précision de la source, virgule française comprise. Ce défaut ne se voyait pas dans les données — le plan visuel était correct — mais uniquement en regardant une image rendue.

Gabarit	Ce qu'il montre
bar_chart	plusieurs valeurs comparées, barres qui poussent, chiffres qui défilent
donut_share	une proportion rapportée à un tout, le reste demeurant visible
ranking_list	un classement où le rang lui-même porte l'information
venn_overlap	une notion contenue dans une autre, ou deux qui se recouvrent
funnel	des étapes qui filtrent, chacune plus étroite que la précédente
pyramid	des niveaux dont la base porte le sommet

Tableau 6.3 — Les gabarits ajoutés et le raisonnement qu'ils rendent visible.



7 Étages 7 et 8 — Images et rendu

7.1 Les arrière-plans

Module : `core/scene_images.py`. Une image d'ambiance est produite par scène, par un modèle de diffusion appelé sur un service public gratuit et sans clé.

Point clé

Ces images sont purement décoratives. Elles sont assombries, floutées et recouvertes par le schéma. Si le service échoue, la scène s'affiche sur un dégradé animé et rien n'est perdu. Cette marginalité est délibérée : elle protège le rendu d'un service sans engagement de disponibilité, et limite l'exposition juridique liée aux productions d'un modèle génératif.

La raison de fond tient à une limite technique des modèles de diffusion : ils ne produisent pas de texte lisible. Toute information portée par l'image serait donc illisible. L'invite interdit explicitement au modèle d'y faire figurer du texte, un chiffre, un logo ou une marque.

`core/scene_images.py` — graine stable

```
def seed_for(scene_id: str) -> int:
    """Python randomise le hachage des chaines a chaque processus, donc
    hash(scene_id) donne une valeur differente a chaque execution --
    exactement l'inverse de ce a quoi sert une graine."""
    digest = hashlib.sha1(scene_id.encode("utf-8")).digest()
    return int.from_bytes(digest[:4], "big") % 100_000
```

Un commentaire qui affirmait le contraire du code

La version précédente utilisait `hash(scene_id)` et le commentaire annonçait « graine déterministe : relancer reproduit la même image ». C'était faux. Mesuré sur trois processus pour la même scène : 20468, 8410, 54640. Chaque exécution téléchargeait un jeu d'images entièrement nouveau.

Un test fige désormais la valeur attendue : si quelqu'un revient à `hash()`, le test tombe.

7.2 Le rendu vidéo

Moteur : Remotion, qui pilote un navigateur sans interface pour dessiner chaque image, puis FFmpeg pour l'encodage. Aucun modèle génératif de vidéo n'intervient.

Point clé

Le rendu est déterministe : deux exécutions du même plan produisent des images identiques. C'est ce qui permet de relancer un chapitre sans obtenir un résultat différent, et de comparer deux versions d'un composant.

Une composition est déclarée par chapitre. Les scènes s'y enchaînent en série, chacune sur une durée calculée à partir de la durée audio mesurée :

remotion/src/components/ChapterComposition.tsx

```
<Series>
  {scenes.map((scene) => {
    const durationInFrames = Math.max(1, Math.round(scene.duration * fps));
    return (
      <Series.Sequence key={scene.scene_id} durationInFrames={durationInFrames}>
        <SceneTransition sceneId={scene.scene_id}>
          <SceneBackdrop sceneId={scene.scene_id} ... />
          <SceneRenderer scene={scene} plan={plan} ... />
        </SceneTransition>
        <Audio src={staticFile(`audio/${scene.scene_id}.wav`)} />
      </Series.Sequence>
    );
  })}
</Series>
<BrandMark />
```

Le répartiteur SceneRenderer sélectionne le composant correspondant à l'archétype planifié et lui transmet les textes. Aucune scène ne demande de code spécifique : ajouter un archétype consiste à écrire un composant, une entrée de vocabulaire et un cas dans le répartiteur.

Trois mécanismes communs à tous les schémas

1. **WrappedText** — le texte SVG ne passe pas à la ligne. Dès que le planificateur a produit des formulations plutôt que des mots isolés, les libellés ont débordé de leurs boîtes. Ce composant replie le texte, réduit le corps si nécessaire, et ne tronque qu'en dernier recours. Les treize schémas d'origine ont dû être repris pour l'utiliser.
2. **TakeawayBand** — la phrase à retenir, portée en bandeau bas par le répartiteur et non par chaque composant, si bien qu'un nouvel archétype l'obtient sans rien faire. Le schéma est décalé de la hauteur du bandeau plutôt que dessiné dessous.
3. **BrandMark** — le logo S2M, tenu en haut à gauche pendant tout le chapitre. Le logo d'origine étant à 69 % brun foncé, une variante claire a été produite : le mot-symbole passe en blanc, le cyan de la marque est conservé.

Un piège du moteur de rendu

AbsoluteFill, le conteneur fourni par Remotion, pose à la fois height:100% et bottom:0 et n'écrase que le second. Le décalage du schéma par le bas restait donc sans effet : la hauteur l'emportait et le diagramme se dessinait en travers du bandeau.

Le conteneur est désormais un div ordinaire avec height: calc(100% - décalage).

8 Étage 9 — Publication

8.1 Le quiz officiel

Module : `core/quiz_reference.py`. Lorsqu'un quiz écrit par les formateurs est fourni, il est lu tel quel plutôt que généré : il reflète ce qu'ils veulent réellement vérifier.

Le document attend une forme simple, un paragraphe par question, et le parseur en extrait l'énoncé, les options et les bonnes réponses.

`core/quiz_reference.py`

```
_OPTION = re.compile(r"(?=\b([A-H])\s)")
_ANSWERS = re.compile(r"Bonnes?\s+R[ée]ponses?\s*:\s*(.+)$", re.I)

# Le document ne fournit aucune justification, et en inventer une
# mettrait des mots dans la bouche du formateur.
return {"question": wording, "options": options,
        "correct_letters": [...], "explanation": ""}
```

Huit questions extraites, toutes vérifiées contre le document. Contrôle fonctionnel : en répondant dans le navigateur avec les lettres du document, le score obtenu est de 8 sur 8.

Une conséquence sur le format

Un quiz officiel porte sur la formation entière, pas sur un chapitre. Le fichier publié passe donc d'un dictionnaire indexé par chapitre à une liste unique, et le lecteur accepte les deux formes. Les questions générées continuent d'afficher leur chapitre d'origine ; les officielles ne l'affichent pas, puisqu'elles n'en ont pas.

8.2 La plateforme de consultation

Le cours est publié dans une plateforme statique — HTML, CSS et JavaScript, sans base de données. La progression, les surlignages et les notes vivent dans le stockage local du navigateur.

- Le catalogue distingue les deux parcours par une pastille, avec pour chacun sa barre de progression.
- Sous la vidéo, la transcription est construite à partir de la piste WebVTT : chaque ligne est horodatée et cliquable, la ligne en cours se surligne et le panneau défile avec la lecture.
- Un double-clic surligne une ligne, un bouton y attache une note ; un onglet rassemble tous les surlignages et notes du cours.

- L'évaluation finale est épinglée en bas du sommaire et porte sur l'ensemble du cours, avec un seuil de réussite à 70 %.
- Le support PDF reste accessible en onglet et en téléchargement.

9 Propriétés du système

9.1 Reproductibilité

Relancer la chaîne sur le même support doit donner le même cours. Deux mécanismes le garantissent, tous deux faciles à casser sans s'en rendre compte.

Le premier est la température, forcée à zéro au point d'entrée unique de tous les appels au modèle, pour qu'un appel ajouté plus tard ne puisse pas l'oublier.

core/llm.py

```
def _generate_content(client: genai.Client, **kwargs):
    config = kwargs.get("config")
    if config is not None and getattr(config, "temperature", None) is None:
        config.temperature = 0.0
    return client.models.generate_content(**kwargs)
```

Ce que coûtait son absence

Sans température fixée, le modèle échantillonne. La décision garder/couper du parcours détaillé, rejouée sur le même enregistrement, a retiré vingt-trois secondes supplémentaires et fait passer un chapitre de 6 min 49 à 6 min 26.

Aucun des deux résultats n'était faux. Mais un système qu'on relance doit répondre deux fois pareil.

Le second est la graine des arrière-plans, décrite plus haut. La limite honnête : les fournisseurs ne garantissent pas une sortie identique au bit près même à température nulle. La variation passe de « un montage différent » à « un mot ici ou là ».

9.2 Résilience

Défaillance	Réaction	Effet
Erreur temporaire (5xx)	nouvelle tentative, délai exponentiel	aucun
Limitation de débit (429)	nouvelle tentative après attente	ralentissement
Quota journalier épuisé	bascule sur le modèle allégé	prose moins dense, signalée
Scène omise par le modèle	second appel ciblé	aucun
Archétype inconnu reçu	repli sur une carte de citation	scène plus sobre
Image non générée	dégradé de couleur	scène sans arrière-plan
Rendu d'un chapitre en échec	les autres se poursuivent	un chapitre à relancer
Interruption de l'exécution	artefacts conservés	reprise à l'étape suivante

Tableau 9.1 — Défaillances anticipées et comportement du système.

Le principe commun est la dégradation progressive : à aucun endroit la défaillance d'un composant accessoire n'interrompt la production.

9.3 Les tests

Vingt-huit tests couvrent les fonctions qui portent les décisions. Ils sont purs — ni réseau, ni modèle, ni clé d'API — et tournent en une trentaine de secondes.

- Détection des pages de séparation, y compris le contre-exemple de la page de clôture.
- Découpage en chapitres : jamais à cheval sur deux sections, équilibré, et proportionné au volume du support.
- Lecture du quiz officiel : énoncé, options et bonnes réponses restitués exactement.
- Diversification : aucun voisin identique, et aucune substitution hors de son groupe de sens.
- Reproductibilité : la graine est figée, la température est bien imposée.
- Rythme des scènes et appariement des sous-titres, hérités du parcours détaillé.

Point clé

Deux vrais défauts ont été trouvés par l'écriture des tests, non par l'observation : le déséquilibre du découpage (202 / 300 / 100 au lieu de trois tiers) et un cas où le nombre de chapitres pouvait dépasser la borne annoncée par la configuration.

10 Limites connues et suites

10.1 Ce qui n'est pas résolu

- Un chapitre écrit par le modèle de repli. Le quota du modèle de rédaction s'est épuisé pendant la production ; le huitième chapitre compte 211 mots contre 280 à 550 pour les autres. Il gagnerait à être refait.
- Des accents perdus dans certains libellés de schémas. Le plan visuel est produit par le modèle allégé, qui écrit parfois « securite » ou « systeme ». Le défaut est visible à l'image sur quelques scènes.
- Un chiffre encore absent : sept des huit valeurs de la diapositive PwC sont citées, « 61 % » ne l'est pas.
- La voix n'est pas celle de l'intervenant. C'est le prix du choix de partir du support ; le parcours détaillé conserve la voix réelle.
- Le service d'images n'offre aucun engagement de disponibilité.
- Validé sur un seul support. Les règles sont écrites pour être générales, mais n'ont été éprouvées que sur ce document.

La limite qui prime sur les autres

Le système produit un cours prêt à être relu, pas prêt à être diffusé. Un titre approximatif ou une nuance perdue ne se corrige pas seul, et la responsabilité d'un contenu de sensibilisation ne peut être imputée à un programme.

Un écran de validation permettant de corriger un titre ou une question avant publication reste l'évolution la plus utile.

10.2 Étendre le système

Ajouter une troisième source — un document Word, une page web, un manuel — demande deux choses et rien d'autre :

1. Un module de lecture qui produit une liste d'objets Scene : un identifiant, un intervalle sur une ligne de temps quelconque, le texte, le chemin d'une image représentative.
2. Un orchestrateur qui enchaîne les étages, en réutilisant ceux de core/.

Tout le reste — plan visuel, arrière-plans, rendu, sous-titres, quiz, publication, les vingt-quatre schémas animés et la plateforme web — fonctionne sans modification. C'est la

propriété que l'architecture cherchait à obtenir, et la production de ce cours en est la première vérification.

10.3 Ajouter un schéma animé

1. Écrire le composant dans `remotion/src/scenes/`. Il reçoit `label`, `items`, `primary`, `secondary` et `durationInFrames`. Tout libellé doit passer par `WrappedText`.
2. L'inscrire au vocabulaire `SCENE_ARCHETYPES` avec une définition d'une ligne, sans référence à un domaine.
3. Ajouter un cas dans le répartiteur `SceneRenderer`.
4. Si le nouveau schéma peut se substituer à un existant, l'ajouter au groupe correspondant de `_INTERCHANGEABLE` pour que la diversification puisse l'utiliser.